
A Small Internal Signal Marks Thinking-Mode Entry in Open Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Some open language models let users turn “thinking” on or off. We ask whether
2 the activation immediately before generation records that choice and whether
3 changing it changes the response branch. For each of the three original fixed-
4 vector models, we average the thinking-minus-off activation difference at the last
5 two prompt tokens over 60 training prompts. On held-out prompts, adding this
6 model-specific vector moves the next-token distribution toward thinking mode
7 in Qwen3.5-4B, Gemma 4 E2B-it, and Nemotron Nano 4B. Exact-copy experi-
8 ments localize the effect to response entry. A Qwen3-4B audit uses the docu-
9 mented soft switch: both modes keep `enable_thinking=True`, use `/think` ver-
10 sus `/no_think`, and receive the same supplied `<think>` token. On 12 localization
11 prompts, copying that token’s activation selects the donor newline on 24/24 direc-
12 tional tests. Controllers of widths 1, 2, 4, and 8 learned on those prompts each
13 select the donor token on 72/72 held-out tests; width 1 is sufficient and has 0/432
14 control wins. Width 8 reaches the switch-text states and one random tensor trans-
15 fers, so it is an extended-context condition. On a separate 12-prompt replication,
16 the copied activation selects one token and is discarded; across three sampling
17 seeds, all 72 continuations follow the donor response format. Across ten seeds and
18 30 held-out non-math prompts, one Nemotron fixed-vector decision produces an
19 exact thinking opening on 293/300 tests versus 0/300 for matched random vectors.
20 Repeated intervention corrupts text. These results identify a causal response-entry
21 signal, not reasoning itself or reliable long-horizon control.

22 1 Introduction

23 Open language models increasingly expose a “thinking” button, a template option, or a system
24 instruction that changes how the model answers. These controls affect latency, cost, response length,
25 and answer style. But a visible `<think>` tag or a long response does not by itself show what changed
26 inside the model.

27 Three things must be kept separate:

- 28 1. the visible text or setting that selects a mode,
- 29 2. the internal activation signal present just before generation, and
- 30 3. the later response, including its length, style, and accuracy.

31 This paper studies the second item. We ask whether the mode choice leaves a measurable internal
32 signal at the end of the prompt and whether changing that signal changes which response style starts.
33 We do not claim that this signal is reasoning itself.

The basic test is simple. For the same user prompt, run the model once with thinking on and once with it off. At the last two prompt tokens, subtract the off-mode activations from the on-mode activations. Average that difference over public training prompts to obtain one model-specific *mode vector*, denoted c_m . Freeze the vector, add it to held-out off-mode activations, and check whether the model’s next-token probabilities move toward those of the held-out thinking run. The intervention sees the held-out off-mode state and the training vector only; the held-out thinking activation is used only afterward for scoring. We call this a *prompt-end mode-signal audit*.

This question is related to token-level reasoning triggers. Mid-Think shows that visible strings such as Okay, </think>, and newline patterns can change response style (Yang et al., 2026). Here the visible switch is held fixed and audited: we directly modify the activation signal it produced. The main result is that several public switches write a small, causal signal at the end of the prompt. The important limitation is equally simple: entering a response mode is not the same as producing a correct or stable long response.

Contributions. This paper makes five contributions.

- We introduce a held-out activation test for whether a public thinking switch writes a recoverable mode signal at the end of the prompt.
- We show that fixed, model-specific mode vectors learned from 60 public paired prompts transfer to held-out prompts in Qwen3.5-4B, Gemma 4 E2B-it, and Nemotron Nano 4B.
- We localize the causal signal to the last prompt tokens in Qwen3.5 and Nemotron. In Qwen3-4B, we use the public soft switch and localize causal control to the shared <think> state immediately before the modes diverge. Controllers learned on 12 prompts transfer on 36 further prompts; a width sweep shows that the one shared position is sufficient. Copying the full post-attention residual state reproduces the donor effect; copying only the MLP update does not reproduce the full effect.
- We test simpler explanations using reversed and random vectors, earlier-token copies, altered suffixes, masked opening tokens, and misleading user instructions.
- We replicate Qwen3’s selected branch token on 12 disjoint, category- and template-matched prompts with three sampling seeds, and test Nemotron generation with a fixed vector on 30 held-out non-math prompts and exact copies on six math prompts. We include negative and degenerate cases and release row-level artifacts and verification scripts.

Table 1: Audit overview. Each row gives one component of the evidence chain.

Audit question	Main evidence	Result signal
Does a public switch leave a reusable internal signal? Where is the signal?	A fixed vector learned on public prompts is added to held-out off-mode activations. Copy the final prompt activations, or Qwen3’s shared branch token; compare with earlier positions.	Next-token probabilities move toward the held-out thinking run. Response-entry copies work; matched earlier copies do not.
Which intermediate state is sufficient?	Copy the full post-attention residual state, or copy only the MLP update.	The former reproduces the donor effect; the latter does not reproduce it fully. This asymmetric test does not isolate the attention update.
Is this only suffix or first-word copying?	Alter suffixes, mask likely opening tokens, reverse the vector, use random vectors, and contradict the mode in user text.	The activation effect survives these controls.
Does it control a full answer?	Generate after one fixed-vector step, after 64 repeated fixed-vector steps, and after reapplying exact activation copies for 64 steps.	One-step branch entry is reliable; the later trajectory is mixed and repeated intervention can corrupt text.

Table 2 separates the four same-checkpoint tests from secondary paired-checkpoint comparisons. This distinction matters: only the same-checkpoint rows study an inference-time switch.

2 Setup

Modes and models. The main fixed-vector tests use three open models with public thinking controls: Qwen3.5-4B (Qwen Team, 2026), Gemma 4 E2B-it (Google DeepMind, 2026), and Llama-3.1 Nemotron Nano 4B v1.1 (NVIDIA, 2025). The new causal audit uses Qwen3-4B (Qwen Team,

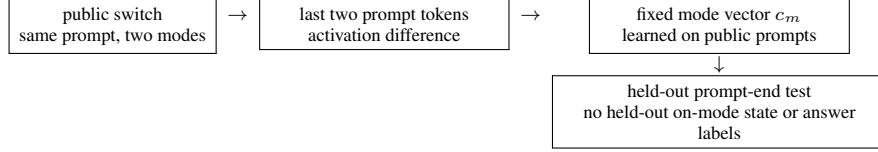


Figure 1: Main protocol. Public paired prompts produce a fixed mode vector. On a held-out prompt, the vector is added to the off-mode activation before the first generated token. The corresponding held-out on-mode activation is not used by the intervention; it is used only to score the resulting probability distribution.

Table 2: What changes between the two modes, what we test, and what we do not infer. “Exact copy” means copying the observed activation from one run into the other; “fixed vector” means adding one average difference learned on other prompts.

Model	How mode is selected	Evidence in this paper	Important limit
Qwen3.5-4B	Same checkpoint; chat-template suffix	Exact copying, fixed vector, broad and contest prompts, controls	One fixed vector is weaker on its 30 broad prompts.
Gemma 4 E2B-it	Same checkpoint; template switch	Fixed vector on broad and contest prompts	Reverse fixed-vector direction is unreliable.
Nemotron Nano 4B	Same checkpoint; system prompt	Exact copying, fixed vector, broad prompts, one-step and sustained generation	One-step entry is reliable; reverse trajectory is incomplete and repeated intervention corrupts text.
Qwen3-4B	Same checkpoint; <code>enable_thinking=True</code> with <code>/think</code> or <code>/no_think</code>	Exact-copy localization; width 1/2/4/8 fixed controllers on 36 held-out prompts; one-token follow-through	The fixed controllers test response entry, not answer accuracy or continued control.
OLMo 3 7B	Separate Think and Instruct checkpoints	Exact copying and fixed-vector comparison	Not an inference-time same-checkpoint switch.
Phi-4 mini / Liquid 1.2B	Separate reasoning or thinking and instruct checkpoints	Secondary exact-copy localization	Not an inference-time same-checkpoint switch; Liquid is asymmetric.

2025). All four compare two modes of the same checkpoint. OLMo 3 7B (OLMo Team, 2026) is analyzed separately because it compares Think and Instruct checkpoints. Earlier Phi and Liquid experiments remain secondary appendix results. In the audited Qwen3.5 template, the assistant suffix differs as follows:

```

think      <|im_start|>assistant\n<think>\n
no-think   <|im_start|>assistant\n<think>\n
           \n</think>\n\n

```

The shared prefix is intentional: in this Qwen3.5 template, no-think opens and immediately closes an empty think block before the answer begins.

Qwen3-4B uses its documented soft switch instead. Both conditions keep `enable_thinking=True`; the user message ends in `/think` or `/no_think`. The two prompts then share the same assistant header, and both modes first emit `<think>`. Their next-token distributions sharply separate: thinking mode’s top token is the single-newline continuation that begins nonempty reasoning, while no-think’s top token is the double-newline continuation of an empty block. We therefore supply the same `<think>` token to both conditions and perform the Qwen3 causal audit at that first divergent decision. This is not the hard template switch obtained with `enable_thinking=False`.

Prompt sets. Table 3 states exactly where prompts come from and how they are used. The fixed vector never sees an AIME prompt during fitting or selection.

Terms used below. An *activation* is the vector of numbers passed between transformer layers. The *residual stream* is this main layer-to-layer path. A *donor* is the run whose mode we copy; the *target* is the run we modify. A *tail patch of width 2* copies activations only at the final two audited tokens. A *fixed mode vector* is one average activation difference learned on other prompts. Appendix G gives intuitive and mathematical definitions together.

The layer scans use different grids. Qwen3.5 tests every residual-stream boundary from 0 through 32. Qwen3 tests boundaries 0, 4, ..., 32, and 36; Nemotron and OLMo test 0, 4, ..., 32. Boundary 0 is

Table 3: Prompt sources and roles. Selection is deterministic; these are not random samples of all possible prompts.

Role	Source and selection	Use
Fixed-vector training	First 30 usable GSM8K test questions and first 30 MATH-500 test rows with a simple numeric final answer	Learn the mean activation difference and choose its scale.
Held-out contest test	All 30 AIME 2025 and all 30 AIME 2026 questions	Prompt text only; no answers, correctness labels, or thinking-mode activations enter the intervention.
Broad transfer	Authored instruction, coding, factual, multi-turn, and non-English prompts; Nemotron also uses GPQA Diamond	Test transfer outside the math training distribution.
Localization and controls	Fixed, category-balanced authored suites; 12 Qwen3-4B localization prompts	Locate the effect and compare post-attention state copies with isolated MLP-update copies.
Qwen3 prompt-value replication	The next two authored instances in each of the same six categories and templates; 12 prompts disjoint from localization	Test whether the selected branch token determines response format under three sampling seeds.
Qwen3 fixed-controller audit	The final two instances in those six categories, plus all 24 prompts from four categories absent from fitting; 36 prompts disjoint from fitting and the prompt-value replication	Test frozen signed controllers of widths 1, 2, 4, and 8 at the shared <think> branch point.

the embedding output passed into the first transformer block. Tables report the first tested boundary where every prompt moves closer to the donor. For Qwen3.5 this is the earliest boundary in a dense scan. For the other models it is only the first successful boundary on a four-layer grid, not an exact onset or a unique causal source.

Throughout the paper, intervention labels are written as *donor mode into target mode*. For example, “think into no-think” means that activations are copied from a thinking run and inserted into the corresponding no-think run before comparing next-token probabilities.

Patch metric. Let p_D and p_T be the donor and target final next-token distributions for the same user prompt under two modes, and let p_P be the distribution after patching target residual state with donor information. We report donor win when

$$JS(p_P, p_D) < JS(p_P, p_T),$$

where JS is Jensen–Shannon divergence, a bounded measure of how different two probability distributions are. A donor win therefore means that the modified run is closer to the intended mode than to its unmodified mode. We also report margin retention for low-rank patches: the fraction of the exact-copy improvement retained by the smaller patch. Confidence intervals resample the stored prompts. They do not cover new checkpoints or all possible prompts, and exact rates such as 60/60 are finite-suite results rather than population guarantees.

Fixed mode vector. The main reusable intervention is one model-specific vector, c_m , that moves a prompt-end activation toward thinking mode:

$$h_{\text{tail}} \leftarrow h_{\text{tail}} + \alpha c_m.$$

For the three original fixed-vector models, c_m is the average thinking-minus-off difference at the final two prompt tokens over 60 training prompts. For Qwen3-4B, separate width-1/2/4/8 mean residual tensors are learned from 12 prompts at boundary 28, added for off-to-think, and subtracted for think-to-off. Each scale is selected on those 12 prompts before evaluating the same 36 held-out prompts. A held-out test uses only the target activation and frozen tensor; the donor activation is used afterward for scoring.

Exact activation copies. For a layer boundary ℓ and width w , an exact tail patch replaces the target activation at the final w prompt positions with the activation observed in the donor run. This is stronger than the fixed-vector test because it uses the matching donor run. Its purpose is localization: matched controls copy the same number of earlier tokens. A separate asymmetric test copies either the full residual state after attention or only the MLP update.

Low-rank residual code. For each prompt i , direction d , and decisive layer ℓ_d , let

$$\Delta_i = \text{vec}(h_{i,\ell_d,\text{tail}}^D - h_{i,\ell_d,\text{tail}}^T).$$

We fit an uncentered SVD basis to $\{\Delta_i\}$ on training prompts, then patch only the projection of each held-out prompt’s exact donor-target difference into the learned subspace. This is a subspace-sufficiency test: at evaluation time, the held-out exact difference supplies the coordinates inside a basis learned only from training prompts. It therefore tests whether the causal mode information lies in a shared low-dimensional subspace, not whether a single fixed steering vector fully determines the branch.

How the fixed vector is fitted. The exact formula for the model- and direction-specific average is

$$\bar{\Delta}_d = \frac{1}{|S_{\text{train}}|} \sum_{i \in S_{\text{train}}} \text{vec}(h_{i,\ell_d,\text{tail}}^D - h_{i,\ell_d,\text{tail}}^T).$$

At held-out evaluation time, we add $\alpha \bar{\Delta}_d$ to the target activation; α is selected on training prompts only. The intervention uses the held-out target activation and the frozen training vector, but no held-out thinking-mode activation or difference. For the cross-family rows, a prompt counts as *effective* only when the modified distribution moves toward the donor by a JS margin above 0.1 and ends within 0.6 JS of it. This prevents tiny movements near the maximum possible distance from counting as successes.

3 The visible switch creates a short-lived internal signal

The first Qwen experiments test whether the public switch is serialized in the chat template and whether the resulting transition states are distinguishable from arbitrary suffix noise. On 60 prompts, the no-think suffix has a deterministic transient path: immediately before the explicit close, the model predicts `</think>` on 60/60 prompts; immediately after the close, it predicts the blank separator on 60/60 prompts. The full no-think suffix then moves to direct-answer openers such as *To*, *Let*, and *Here*, while think mode moves to thinking-preface openers such as *Thinking* and *Here*.

Matched controls separate the complete template sequence from generic sensitivity to a changed suffix. On 180 Qwen prompts spanning ten categories, a real `</think>` outside the empty-block sequence is close to no-think but not identical (mean JS 0.0534). Fake close tags and moved-close controls do not reproduce the canonical mode. Same-length *Okay*, *Think*, and whitespace suffixes also remain far from thinking mode (mean JS 0.6519, 0.6767, and 0.6618). The full serialized sequence matters; merely changing the final tokens is insufficient.

Three further audits test simpler explanations. First, Qwen’s fixed-vector suite includes calendar, coding, dynamic programming, language, logic, and planning prompts as well as math. On the non-math categories, it reaches donor win 102/108 for no-think into think and 94/108 in reverse; on math, counting, and probability it reaches 72/72 in both directions.

Second, Table 4 removes likely mode-signaling tokens before computing JS. The fixed mode vector remains 10/10 in both directions after removing special and whitespace tokens, common thinking/direct openers, and a broader hand-written list. A verbose-versus-concise style vector falls to chance. With a learned mask, the no-think-into-think vector remains 9/10 after removing the 10 most mode-predictive tokens, 8/10 after 100, and 9/10 after 250. Three norm-matched random vectors never exceed 3/10 and move in the wrong direction on average.

Third, user instructions are made deliberately misleading. Across 10 held-out prompts, seven prompt variants, and both modes, a classifier using the final activation identifies the actual template mode in 140/140 cases. A heuristic using the visible instruction is correct in 60/140; a heuristic using the first predicted token is correct in 107/140. The signal is therefore not explained by an ordinary request for verbosity or a single opening token.

4 The causal signal is present at response entry

Table 5 reports exact activation-copying tests. In Qwen3.5 and Nemotron, copying the final two prompt-token activations makes every tested prompt closer to the copied mode at a tested layer, while copying two earlier positions never wins. Qwen3 requires a slightly different audit because the public soft switch always emits a `<think>...</think>` wrapper. We keep `enable_thinking=True`, use `/think` versus `/no_think`, supply the same first `<think>` token to both runs, and audit the next

Table 4: Qwen token-mask audit. Each cell is held-out donor wins and JS margin after removing the listed tokens and renormalizing. The fixed mode vector survives; a visible verbosity vector does not.

Direction	Method	Unmasked	Mask openers	Mask branch markers
think into no-think	branch vector	10/10, .571	10/10, .455	10/10, .455
think into no-think	verbosity vector	7/10, .037	5/10, .044	5/10, .044
no-think into think	branch vector	10/10, .064	10/10, .108	10/10, .108
no-think into think	verbosity vector	6/10, -.001	5/10, -.000	5/10, -.000

171 decision. Copying the activation at that one shared token produces donor wins on 12/12 prompts in
172 both directions; copying the matched user-end positions four tokens earlier or the first four prompt
173 positions produces 0/12. These exact-copy tests use the matching donor run and are separate from
174 the stronger fixed-vector transfer claim. OLMo is shown separately because it compares two check-
175 points. Phi and Liquid checks remain secondary appendix results.

Table 5: Exact copying at response entry. Qwen3.5, Nemotron, and OLMo copy the final two prompt positions; Qwen3 copies the single shared <think> state at the soft switch’s first divergent decision. A successful tested boundary has donor win on every prompt in that finite suite.

Model	Donor into target	Copied width	First success	Earlier control
<i>Same-checkpoint public switch</i>				
Qwen3.5-4B	think into no-think	2	3	0.000 best donor win
Qwen3.5-4B	no-think into think	2	0	0.000 best donor win
Qwen3-4B soft	think into no-think	1	24	0.000 best donor win
Qwen3-4B soft	no-think into think	1	28	0.000 best donor win
Nemotron Nano 4B	think into off	2	16	0.000 best donor win
Nemotron Nano 4B	off into think	2	12	0.000 best donor win
<i>Paired-checkpoint comparison</i>				
OLMo 3 7B	think into instruct	2	0	0.000 best donor win
OLMo 3 7B	instruct into think	2	4	0.000 best donor win

176 After localizing the Qwen3 soft-switch decision, we fit controllers of widths 1, 2, 4, and 8 on the
177 same 12 prompts at boundary 28. Each selects the donor token on 72/72 held-out directional tests.
178 Width 1 is sufficient and has 0/432 control wins. Wider deterministic controls also have zero wins;
179 one width-8 random tensor wins on 21/36 prompts because that span reaches the switch instruction.
180 No held-out donor activation or difference enters any controller. Appendix B gives every denomina-
181 tor.

182 The module-labelled tests are asymmetric. The historical `attention_only` condition copies the
183 donor’s full residual state after attention, including the donor layer input, and then runs the MLP.
184 The `mlp_only` condition keeps the target’s post-attention state and copies only the donor MLP
185 update. In Qwen3.5-4B, the post-attention-state copy reaches donor win 1.000 in both directions;
186 the MLP-update copy reaches 0.000 in one direction and at most 0.167 in the other. At Qwen3-4B’s
187 soft-switch branch point and in Nemotron, post-attention-state copies reach 1.000 in both directions
188 while MLP-update copies remain 0.000. Thus the post-attention state reproduces the full effect
189 while the isolated MLP update does not reproduce it fully. This does not isolate the attention update
190 or one attention head.

191 5 How the signal is written and read

192 Three tests give a simple picture. First, the signal is present at response entry: exact final-prompt
193 copies work in Qwen3.5 and Nemotron, and both exact copying and held-out fixed controllers work
194 at Qwen3-4B’s shared soft-switch branch token. Second, Qwen3 width 1 succeeds on 72/72 held-
195 out tests at the branch point and 0/72 at a non-overlapping earlier position; larger widths do not
196 improve it. Nemotron succeeds on 42/42 broad-plus-GPQA prompts at the final two positions and
197 0/42 when moved 8 or 16 tokens earlier. Third, the post-attention residual copy reproduces the

Table 6: Fixed-vector selection. The grid and rule are fixed before the held-out AIME prompt test. Effective requires JS margin > 0.1 and JS-to-donor < 0.6 .

Model	Direction	Width grid	Fixed layer	Alpha grid / selected
Qwen3.5-4B	think into no-think	{2}	15	{.25,.5,1,2,4,8} / 1
Qwen3.5-4B	no-think into think	{2}	19	{.25,.5,1,2,4,8} / 2
Gemma 4 E2B-it	think into off	{2}	35	{.25,.5,1,2,4,8} / 2
Gemma 4 E2B-it	off into think	{2}	35	{.25,.5,1,2,4,8} / 1
Nemotron Nano 4B	think into off	{2}	28	{.25,.5,1,2,4,8} / 4
Nemotron Nano 4B	off into think	{2}	28	{.25,.5,1,2,4,8} / 2

Table 7: Held-out contest-prompt test for same-checkpoint fixed vectors. Each vector is learned on 60 paired public prompts and tested on all 60 AIME 2025+2026 question strings. It uses no AIME answers, correctness labels, thinking-mode activations, or mode differences.

Model	Entry path	Think eff.	Think JS	Opp. eff.	Random eff.
Qwen3.5-4B	template suffix	60/60	0.1103	0/60	19/180
Gemma 4 E2B-it	template switch	60/60	0.0002	0/60	0/180
Nemotron Nano 4B	system prompt	60/60	0.0000	0/60	0/180

donor effect while the isolated MLP update does not reproduce it fully; because these interventions are asymmetric, they do not attribute the signal to attention alone.

6 Transfer beyond the training prompts

The main non-AIME test learns fixed vectors on 60 public GSM8K/MATH-500 prompts and evaluates them on instructions, coding, factual questions, multiturn conversations, non-English prompts, and GPQA Diamond. Gemma succeeds on 30/30 broad prompts (JS-to-donor 0.0204; random vectors 0/90). Nemotron succeeds on 30/30 (JS-to-donor 0.0001; random 0/90), and a reduced broad-plus-GPQA test succeeds on 42/42; moving the same vector to earlier positions gives 0/42. Qwen3-4B’s width-1 branch controller succeeds on 72/72 held-out directional tests, including 48/48 in four categories absent from fitting; widths 2, 4, and 8 do not improve this maximum. Qwen3.5’s broader width-2 fixed vector transfers less consistently; Section 11 reports that boundary.

7 Held-out contest-prompt test

We also test all 60 AIME 2025 and 2026 questions as a disjoint source of prompt text; we do not generate or grade their answers. Width is fixed at 2 from the earlier localization test. Layer is fixed before held-out evaluation. The scale α is chosen on the 60 training prompts by prioritizing the number of effective prompts, then donor wins, JS margin, and JS-to-donor. No AIME prompt, answer, thinking-mode activation, or mode difference enters that selection. Table 7 then adds the frozen vector to each held-out off-mode activation.

The three model-specific vectors succeed on 180/180 held-out prompt boundaries. Reversing their signs gives 0/180. Three norm-matched random vectors per model give 19/540, all in the noisier Qwen3.5 row. These are prompt-boundary mode results, not AIME accuracy results.

Secondary OLMo checkpoint-pair result. OLMo 3 7B compares separately trained Think and Instruct checkpoints. The same fixed-vector protocol reaches 60/60 held-out prompts toward Think (mean JS-to-donor 0.0207) and 60/60 toward Instruct (0.2873). This secondary result shows that the test can find an analogous activation difference across checkpoints; it is not evidence about a same-checkpoint public switch.

To summarize the leakage controls: width, layer, and the effective threshold are fixed before the held-out test; only α is selected on training prompts. AIME prompts are disjoint from training, and neither their answers nor their thinking-mode activations are used.

Table 8: Cross-validated low-rank residual subspace results. CIs are prompt-bootstrap intervals. Random subspaces of matched rank achieve donor win 0.000 in all rows.

Model	Donor into target	Rank	Donor win	Margin retention
Qwen3.5-4B	think into no-think	2	1.000 [1.000, 1.000]	0.966 [0.906, 1.046]
Qwen3.5-4B	no-think into think	2	1.000 [1.000, 1.000]	0.910 [0.869, 0.948]
OLMo 3 7B	think into instruct	2	1.000 [1.000, 1.000]	0.862 [0.811, 0.908]
OLMo 3 7B	instruct into think	1	1.000 [1.000, 1.000]	1.021 [0.963, 1.074]

8 Low-rank support

The fixed-vector audit above is the main deployable prompt-boundary test. A separate low-rank subspace analysis asks a more mechanistic question: whether the exact held-out mode difference lies in a shared low-dimensional causal subspace. Table 8 gives that result. We run 3-fold category-balanced cross-validation over a 60-prompt suite; each fold trains on 40 prompts and evaluates on 20 held-out prompts. The learned spectrum is concentrated in Qwen, where the top two components explain 0.874 and 0.854 of donor-target tail-difference energy in the two directions. OLMo has a similarly compressed paired-checkpoint subspace. The subspace interventions transfer causally; unlike the fixed-vector audit, they use the held-out prompt’s exact donor-target difference to choose coordinates inside the train-learned subspace, so they support the low-rank mechanism rather than fixed-vector deployability.

A larger Qwen-only 180-prompt subspace suite clarifies what the code represents. Full width-2 patches have donor win 1.000 with JS to donor near zero. Rank-2 SVD patches also reach donor win 1.000 in both directions; matched random rank-2 subspaces remain at donor win 0.000. Wrong-prompt exact-difference patches transfer strongly, reaching donor win 0.956 and 0.994 in the two directions. The branch code is therefore shared rather than prompt-keyed, consistent with the fixed-vector result above.

Logit attribution of the rank-2 Qwen code is branch-vocabulary-shaped. The first component in the think-donor direction promotes tokens such as `Thinking`, `thinking`, and `user-analysis` tokens, while suppressing direct-answer openers such as `To`, `To`, and `###`. This ties the residual subspace to the visible response program rather than leaving it as an abstract donor-win vector.

9 Generated behavior diagnostics

On a separate 12-prompt value replication, one Qwen3 exact copy selects the donor newline on all 24 directional tests. We discard the copied state, insert only that token, and sample under three seeds: all 72 continuations follow the donor format, while all 72 paired target controls retain the target format. Every generation completes with one closed think block and a nonempty answer. This tests one-token format mediation, not accuracy or residual rollout.

Across ten settings on 30 frozen non-math prompts, one Nemotron fixed-vector decision produces an exact `<think>` opening on 293/300 generations versus 0/300 for norm-matched random vectors; the two-way prompt-and-seed bootstrap interval is [0.930, 1.000]. The fixed next-token distribution wins on every prompt in both directions. A separate 1,024-token test finds no fixed-vector repetition, but reverse outputs remain far from canonical off. Entry selection is reliable; full-response control is not.

Applying the same Nemotron vector for 64 decisions corrupts all 30 prompts in both directions. On six math prompts, we instead recompute and reapply matching donor tail activations for each of the first 64 decisions, then continue freely. Donor resemblance still declines, and three of 24 modified runs remain repetitive at the 8,192-token limit. Appendix F gives full protocols, tables, and additional Qwen diagnostics.

10 Related work

Hybrid-reasoning work studies when to spend reasoning budget and how visible triggers or steering vectors change response style (Jiang et al., 2025; Yang et al., 2026; Venhoff et al., 2025). We instead test the internal prompt-end signal written by a public switch, separating exact copies from vectors learned on other prompts. Causal-rationale work studies later reasoning traces (Rahim and Rahim, 2025; Sathyanarayanan et al., 2026); we study the state before a long trace exists. We build on activation patching and layerwise readout (Geva et al., 2023; Belrose et al., 2023; Zhang and Nanda, 2024), adding reversed, random, earlier-position, token-mask, suffix, and misleading-instruction controls.

11 Limitations

We measure response entry, not faithful or correct reasoning. The AIME rows compare next-token distributions from question text without generating or grading answers; accuracy appears only in separate generation diagnostics.

The fixed vectors are model-specific. Qwen3.5, Gemma, and Nemotron use width-2 prompt-tail vectors. Qwen3-4B succeeds at widths 1, 2, 4, and 8 over 36 held-out prompts, but all test only the first branch decision. Width 1 is sufficient; width 8 reaches switch-text states and has a nonzero random control. Qwen3’s separate format experiment discards the copied state after one token. Neither result establishes accuracy or continued control. OLMo compares separate checkpoints. Gemma’s reverse direction works under exact copying but not reliably under one fixed vector.

OLMo, Phi, and Liquid tests compare paired checkpoints, not public same-checkpoint switches; Phi succeeds both ways on eight prompts while Liquid succeeds only from instruct into think. Models without paired modes require a different experiment.

Transfer varies: Gemma and Nemotron succeed on broad prompts, while Qwen3.5’s fixed vector meets the strict threshold on 13/30 broad prompts versus 2/90 random controls despite strong localization and contest-prompt transfer.

Generated answers are harder because each sampled token changes what follows. Qwen3’s one-token intervention changes format and length but does not test accuracy or modify later activations. One Nemotron step reliably selects the opening on 30 held-out non-math prompts; forward outputs resemble canonical thinking, but reverse outputs do not recover canonical direct mode. Reapplying the fixed vector for 64 steps corrupts every output. Recomputing and reapplying exact donor tail activations for 64 steps still loses donor resemblance, with three of 24 runs repetitive at the 8,192-token limit. Reliable full-answer residual control remains unsolved.

Localization remains partial: Qwen3.5 and Nemotron point to final prompt positions, and Qwen3 to its shared pre-divergence state. A full post-attention residual copy works while an isolated MLP update does not, but this isolates neither attention nor one head. We find no cross-family vector or benchmark-solving policy; visible controls still initiate the mode, and proprietary switches may differ.

12 Conclusion

Public switches in four same-checkpoint models leave causal entry signals. Qwen3-4B controllers of widths 1, 2, 4, and 8 each give 72/72 held-out donor tokens; width 1 is sufficient and has 0/432 control wins. An exact copy sets format on 72/72 continuations. Fixed vectors also transfer in Qwen3.5, Gemma, and Nemotron, whose audit gives 293/300 exact thinking openings versus 0/300 random controls. Repeated intervention fails: this is response entry, not reasoning or full-answer control.

References

Nora Belrose, Zach Furman, Logan Smith, Danny Halawi, Igor Ostrovsky, Lev McKinney, Stella Biderman, and Jacob Steinhardt. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*, 2023.

313 Mor Geva, Jasmijn Bastings, Katja Filippova, and Amir Globerson. Dissecting recall of factual
314 associations in auto-regressive language models. In *EMNLP*, 2023.

315 Lingjie Jiang, Xun Wu, Shaohan Huang, Qingxiu Dong, Zewen Chi, Li Dong, Xingxing Zhang,
316 Tengchao Lv, Lei Cui, and Furu Wei. Think only when you need with large hybrid-reasoning
317 models. *arXiv preprint arXiv:2505.14631*, 2025.

318 Google DeepMind. Gemma thinking capabilities documentation and Gemma 4 E2B-it model card.
319 <https://ai.google.dev/gemma/docs/capabilities/thinking>; <https://huggingface.co/google/gemma-4-E2B-it>, 2026.

321 Liquid AI. LFM2.5 1.2B Thinking model card. [https://huggingface.co/LiquidAI/LFM2.5](https://huggingface.co/LiquidAI/LFM2.5-1.2B-Thinking)
322 -1.2B-Thinking, 2026.

323 Microsoft. Phi-4-mini-reasoning model card. [https://huggingface.co/microsoft/Phi-4-m](https://huggingface.co/microsoft/Phi-4-mini-reasoning)
324 ini-reasoning, 2025.

325 NVIDIA. Llama-3.1-Nemotron-Nano-4B-v1.1 model card. [https://huggingface.co/nvidia/](https://huggingface.co/nvidia/Llama-3.1-Nemotron-Nano-4B-v1.1)
326 Llama-3.1-Nemotron-Nano-4B-v1.1, 2025.

327 OLMo Team. OLMo 3 7B Think model card. [https://huggingface.co/allenai/Olmo-3-7](https://huggingface.co/allenai/Olmo-3-7B-Think)
328 B-Think, 2026.

329 Qwen Team. Qwen3.5-4B model card. <https://huggingface.co/Qwen/Qwen3.5-4B>, 2026.

330 Qwen Team. Qwen3-4B model card. <https://huggingface.co/Qwen/Qwen3-4B>, 2025.

331 Noor Rahim and Ali Abdul Rahim. Dormant reasoning circuits in RL-trained language models. In
332 *FoRLM*, OpenReview, 2025. <https://openreview.net/forum?id=mTjGBrkdztz>.

333 Anish Sathyanarayanan, Aditya Nagarsekar, and Aarush Rathore. Bypassing the rationale: Causal
334 auditing of implicit reasoning in language models. *arXiv preprint arXiv:2602.03994*, 2026.

335 Constantin Venhoff, Iván Arcuschin, Philip H. S. Torr, Arthur Conmy, and Neel Nanda. Understand-
336 ing reasoning in thinking language models via steering vectors. *arXiv preprint arXiv:2506.18167*,
337 2025.

338 Wang Yang, Debargha Ganguly, Xinpeng Li, Chaoda Song, Shouren Wang, Vikash Singh, Vipin
339 Chaudhary, and Xiaotian Han. Mid-Think: Training-free intermediate-budget reasoning via token-
340 level triggers. *arXiv preprint arXiv:2601.07036*, 2026.

341 Fred Zhang and Neel Nanda. Towards best practices of activation patching in language models:
342 Metrics and methods. In *ICLR*, 2024.

343 A Artifact map

344 The row-level artifacts are stored under `artifacts/`. Artifact directories contain run metadata plus
345 aggregate and row-level files where applicable. For double-blind review, the submitted supplement
346 should be built with `scripts/prepare_neurips_supplement.sh`; this script copies repro-
347 duction scripts, selected row-level and aggregate artifacts, prompt splits, and sanitized run metadata
348 while excluding paper source, written appendices, raw upload logs, and machine-specific shell logs.

349 The supplement also includes `scripts/verify_claim_artifacts.py`, which checks that the
350 artifact directories and primary files in the artifact map below are present, and `scripts/verify_h`
351 `eadline_numbers.py`, which checks the paper’s rounded headline values against the stored JSON
352 artifacts.

Claim	Artifact directory	Primary aggregate files
Qwen tail localization	<code>qwen35-paper-interp-20260422T075824Z</code>	<code>width_patch_aggregate.json</code>
Qwen3.5 post-attention/MLP comparison	<code>qwen35-controller-behavior-20260422T184734Z</code>	<code>module_patch_aggregate.json</code> , <code>module_patch_rows.json</code> , <code>SUMMARY.md</code>

Claim	Artifact directory	Primary aggregate files
Historical Qwen3-4B hard-template width sweep	rebuttal-20260724/qwen3-4b-localization	patch_aggregate.json, first_perfect_layers.json, template_serialization.json, SUMMARY.md
Qwen3-4B soft-switch prompt-tail width sweep	rebuttal-20260725/qwen3-4b-soft-switch-localization	patch_aggregate.json, first_perfect_layers.json, SUMMARY.md
Qwen3-4B public soft-switch audit	rebuttal-20260725/qwen3-4b-soft-switch-branch-point	patch_aggregate.json, module_patch_aggregate.json, first_perfect_layers.json, SUMMARY.md
Qwen3-4B same-prompt one-token diagnostic	rebuttal-20260727/qwen3-4b-soft-switch-generation	generation_aggregate.json, generation_rows.json, prompt_suite.json, SUMMARY.md
Qwen3-4B template-matched one-token replication	rebuttal-20260727/qwen3-soft-switch-one-token-followthrough	branch_rows.json, generation_aggregate.json, generation_rows.json, prompt_suite.json, SUMMARY.md
Qwen3-4B held-out fixed controller	rebuttal-20260728/qwen3-soft-switch-fixed-controller	audit_aggregate.json, audit_rows.json, alpha_selection.json, controller_diagnostics.json, SUMMARY.md
Qwen3-4B held-out fixed-controller width sweep	rebuttal-20260728/qwen3-soft-switch-fixed-controller-width-1-2-4-8	audit_aggregate.json, audit_rows.json, alpha_selection.json, token_span_audit.json, SUMMARY.md
Qwen token controls and 180 prompts	qwen35-neurips-hardening-20260425T172250Z	token_control_aggregate.json, crossval_aggregate.json
Qwen fixed steering vector	qwen35-fixed-controller-20260427T152833Z	fixed_prompt_controller_aggregate.json, fixed_prompt_controller_bootstrap_summary.json
Qwen generated-prefix rollout diagnostic	qwen35-multilayer-controller-benchmark-20260427T233149Z	generation_aggregate.json, generation_rows.json
Gated public-task steering policy	gated-controller-policy-20260505T050337Z	gated_policy_aggregate.json, gated_policy_aggregate_by_model.json, gated_policy_rows.json, crossvalidated_gated_policy_rows.json, gated_policy_paired_deltas.json
Cross-family fixed vectors	cross-family-fixed-controller-20260427T195227Z	fixed_controller_aggregate.json, fixed_controller_rows.json
Contest-style prompt-string branch-state audit	aime-fixed-branch-controller-20260429T194405Z	aime_controller_aggregate.json, aime_controller_rows.json, controller_vectors.json
Gemma/Nemotron fixed thinking-branch switch	gemma-nemotron-fixed-switch-20260505T070908Z	aime_controller_aggregate.json, aime_controller_rows.json, controller_selection.json, controller_vectors.json
Broad branch-state claim synthesis	branch-state-broad-claim-20260506T034229Z	branch_state_broad_claim_summary.json, SUMMARY.md
Branch-entry robustness audits	branch-entry-paper-audits-20260506T210000Z	paper_audit_summary.json, SUMMARY.md, svd_top_token_proxy.json
Qwen masked-JS/style control audit	qwen35-masked-js-style-control-20260506T215634Z	masked_js_style_aggregate.json, masked_js_style_rows.json, mask_metadata.json
Qwen learned branch-token mask audit	qwen35-learned-token-mask-20260507T040000Z	learned_mask_aggregate.json, learned_mask_rows.json, learned_branch_tokens_top250.json
Qwen adversarial branch audit	qwen35-adversarial-branch-audit-20260507T000000Z	classifier_aggregate.json, patch_aggregate.json, SUMMARY.md
Nemotron broad+GPQA robustness	prompt-boundary-robustness-nemotron-reduced-20260507T061500Z	fixed_aggregate.json, random_direction_summary_aggregate.json, subsets_aggregate.json, wrong_position_aggregate.json
Historical direct-erasure adapter diagnostics	shared-latent-step-direct-erasure-20260430T040633Z	shared_latent_aggregate.json, shared_latent_rows.json, latent_adapters.json
Historical Nemotron adapter stress test	shared-latent-nemotron4b-direct-erasure-step-20260505T040225Z	shared_latent_aggregate.json, shared_latent_rows.json, latent_adapters.json
Closed-loop direct-erasure diagnostic	closed-loop-shared-direct-erasure-reduced-20260505T041600Z	closed_loop_step_aggregate.json, closed_loop_prompt_aggregate.json, closed_loop_prompt_rows.json
Historical Phi kernel induction diagnostic	shared-latent-step-thinking-phi-kernel-layer28-20260430T052209Z	shared_latent_aggregate.json, shared_latent_rows.json, latent_adapters.json
OLMo localization	olmo3-localization-20260423T160830Z	block_patch_aggregate.json, module_patch_aggregate.json
Nemotron localization	nemotron-localization-20260507T000000Z	patch_aggregate.json, module_patch_aggregate.json, first_perfect_layers.json
Nemotron focused generation audit	rebuttal-20260724/nemotron-combined-analysis	combined_generation_aggregate.json, combined_generation_rows.json, fidelity_by_step.json

Claim	Artifact directory	Primary aggregate files
Nemotron one-step fixed-vector generation	rebuttal-20260724/nemotron-fixed-vector-one-step-generation	generation_aggregate.json, generation_rows.json, controller_step_rows.json, PAIRED_QUALITY_ANALYSIS.md
Nemotron 64-step stress test	rebuttal-20260724/nemotron-fixed-vector-broad-generation	generation_aggregate.json, generation_rows.json, controller_step_rows.json
Nemotron ten-seed entry audit	rebuttal-20260727/nemotron-ten-seed-entry-analysis and rebuttal-20260727/nemotron-ten-seed-entry-runs	analysis.json, SUMMARY.md, per-seed_generation_rows.json and controller_step_rows.json
Liquid localization	liquid-causal-patch-20260426T044355Z, liquid-causal-layer-scan-20260430T051648Z	patch_aggregate.json, canonical_aggregate.json
Phi localization	phi-causal-patch-20260423T231120Z	patch_aggregate.json
Low-rank cross-validation	residual-subspace-crossval-20260424T205031Z	crossval_aggregate.json, spectrum_aggregate.json
Public behavior diagnostics	qwen35-public-benchmark-behavior-20260424T104510Z	aggregate_by_variant.json
Sustained patch generation	qwen35-sustained-patch-generation-20260425T193507Z	generation_aggregate.json, sustained_step_aggregate.json

B Qwen3 width sweeps and fixed-controller audit details

B.1 Earlier exact-copy width sweeps

Two earlier 12-prompt experiments tested widths 1, 2, 4, and 8 using the matching donor activation for each prompt. Table 10 reports them because they motivated the cleaner branch-point design, not because they establish held-out controller transfer. The first used the hard chat-template switch, `enable_thinking=True` versus `False`; its serialized prompts differ by a four-token assistant suffix. The second kept `enable_thinking=True` and used `/think` versus `/no_think`, but patched the ordinary serialized prompt tail before adding the shared `<think>` token. In that soft-switch sweep, non-tail width-2/4/8 copies also reach donor win 1.0, so it does not uniquely localize the effect to the tail. The first four prompt positions never win in either direction.

Switch and copied block	Direction	Width 1	Width 2	Width 4	Width 8
Hard template, tail	off to think	20	0	0	0
Hard template, tail	think to off	28	4	0	8
Hard template, non-tail	off to think	—	none	none	none
Hard template, non-tail	think to off	—	none	none	none
Soft instruction, tail	off to think	32	28	24	0
Soft instruction, tail	think to off	32	24	24	0
Soft instruction, non-tail	off to think	—	8	0	0
Soft instruction, non-tail	think to off	—	8	0	0

Table 10: Earlier Qwen3-4B exact-copy width sweeps. Entries are the first tested residual boundary with donor wins on all 12 prompts; the grid is 0, 4, . . . , 36, “none” means no tested boundary, and “—” means the block was not tested. Every tail width reaches 12/12 at some boundary in both directions. These are same-prompt donor copies, not fixed controllers, and have no fit/held-out split.

B.2 Held-out fixed-controller width sweep

The new width protocol, runner, prompt IDs, layer boundary, alpha grid, selection rule, controls, random seeds, and interpretation limits were committed before inference. Both conditions use model revision 4dcb3d101c2a062e5c1d4bb173588c54ea6c4d25, keep `enable_thinking=True`, and differ only through the public `/think` and `/no_think` instructions. We supply the same `<think>` token, then audit the first divergent next-token decision. For each width, the frozen controller is the mean thinking-minus-off residual tensor over the same 12 fit prompts at boundary 28. The fit-only selection rule chooses $\alpha = 1$ for every width. Evaluation uses 12 new values from the same six authored categories and 24 prompts from four categories absent from fitting. It never uses a held-out donor activation or difference.

Width 1 already reaches the maximum binary result, so more positions add no success benefit. Its mean JS-to-donor is 8.05×10^{-9} off-to-think and 9.94×10^{-7} think-to-off; mean donor margins are

Width	α	Fixed wins	Token matches	Zero	Reversed	Earlier span	Random
1	1	72/72	72/72	0/72	0/72	0/72	0/216
2	1	72/72	72/72	0/72	0/72	0/72	0/216
4	1	72/72	72/72	0/72	0/72	0/72	0/216
8	1	72/72	72/72	0/72	0/72	0/72	21/216

Table 11: Held-out Qwen3-4B fixed-controller width sweep over 36 prompts and both directions. Every fixed row comprises 24 same-category and 48 category-shift directional tests. The earlier-span control has the same width but never overlaps the active response-entry span. Random denominators contain three preregistered norm-matched tensors. All 21 random wins are one width-8 seed in the off-to-think direction.

0.69315 and 0.69313. Controller norms for widths 1/2/4/8 are 112.587/144.613/201.942/303.778, and the corresponding mean fit-prompt cosines to the mean tensor are 0.975/0.951/0.963/0.960. The tokenizer audit shows that widths 1, 2, and 4 stay within the shared <think> and assistant-header suffix, while width 8 reaches the differently tokenized /think and /no_think instruction fragments. This explains why width 1 is the cleanest sufficiency and specificity result. These tests do not measure accuracy, reasoning quality, or later-token control.

C Localization effect sizes

Table 12 expands the headline localization rows from Table 5. The Qwen3.5 JS columns are point estimates from the stored aggregate. Qwen3, Nemotron, OLMo, Liquid, and Phi include prompt-bootstrap intervals. Qwen3 reports the shared-token soft-switch decision after both modes receive <think>. Phi reports the main 8-prompt set; the alternate set reaches the same first successful tested boundaries.

Model	Donor into target	n	Layer	Donor win	JS to donor	JS to target
Qwen3.5-4B	think into no-think	60	3	1.000 [1.000, 1.000]	0.0632	0.6283
Qwen3.5-4B	no-think into think	60	0	1.000 [1.000, 1.000]	0.1113	0.6080
Qwen3-4B soft	think into no-think	12	24	1.000 [1.000, 1.000]	0.0000001	0.6931
Qwen3-4B soft	no-think into think	12	28	1.000 [1.000, 1.000]	0.0000002	0.6931
Nemotron Nano 4B	think into off	12	16	1.000 [1.000, 1.000]	0.1404	0.6893
Nemotron Nano 4B	off into think	12	12	1.000 [1.000, 1.000]	0.1222	0.6748
OLMo 3 7B	think into instruct	12	0	1.000 [1.000, 1.000]	0.2552	0.6925
OLMo 3 7B	instruct into think	12	4	1.000 [1.000, 1.000]	0.5022	0.6927
Liquid LFM2.5 1.2B	think into instruct	60	16 best	0.000 [0.000, 0.000]	0.6924	0.6037
Liquid LFM2.5 1.2B	instruct into think	60	12	1.000 [1.000, 1.000]	0.1246	0.6931
Phi-4 mini	think into instruct	8	28	1.000 [1.000, 1.000]	0.0110	0.6932
Phi-4 mini	instruct into think	8	16	1.000 [1.000, 1.000]	0.2996	0.6921

Table 12: Effect sizes for the response-entry localization rows. Qwen3 uses a width-1 copy of the shared <think> state; the other headline rows use width 2. Qwen3, Nemotron, and OLMo boundaries are the first successes on coarse four-layer grids, not exact onsets. The Liquid think-into-instruct row reports the best tested tail boundary because no tested boundary produced donor wins. Matched non-tail controls have donor win 0.000 [0.000, 0.000] for all rows with bootstrap intervals in the artifacts.

D Compute and existing assets

Experiments were run on an Apple M4 Mac Mini with 16GB unified memory using MLX and MLX-LM. The BF16 Qwen replication covers fewer configurations than the 4-bit cross-validation because of this memory limit. The sustained-generation suite is also smaller because it recomputes full donor and target traces at each generated step. The supplement’s per-run run_metadata.json files record UTC start times, platform strings, model identifiers, layer and alpha grids, prompt counts, and generation caps for the included runs.

The model checkpoints are open Hugging Face assets: Qwen3.5-4B and Qwen3-4B are Apache-2.0, OLMo 3 7B Think/Instruct is Apache-2.0 with Ai2 Responsible Use Guidelines, Gemma 4 E2B-it is documented by Google DeepMind with thinking-mode controls, LiquidAI LFM2.5 1.2B uses the LFM Open License v1.0, Phi-4-mini-reasoning is MIT, and Llama-3.1-Nemotron-Nano-

4B-v1.1 uses the NVIDIA Open Model License with additional Llama 3.1 terms. The prompt-boundary branch audits use math-ai/aime25 and math-ai/aime26, which are Apache-2.0 licensed on their Hugging Face dataset pages. Public generation diagnostics are provided as behavior artifacts for the same intervention family.

402 E Broader impacts

403 This work gives a concrete audit target for hybrid-reasoning systems: an activation measured before
404 generation is easier to test than a vague claim that thinking is on. The main dual-use risk is superficial
405 manipulation of thinking-style behavior, so we pair the result with held-out tests and failures that
406 distinguish mode entry from answer quality.

407 F Generated behavior diagnostics

408 F.1 Qwen3 template-matched one-token replication

409 The follow-through suite freezes every choice made by the 12-prompt localization audit, then
410 uses the next two authored instances from each of its six categories and templates. The result-
411 ing 12 prompts are disjoint from localization but category- and template-matched. This tests new
412 prompt values within the same authored structures, not broad-distribution transfer. Both modes keep
413 `enable_thinking=True`, use `/think` or `/no_think`, and receive the same supplied `<think>` to-
414 ken. The copied residual state is used only to choose the next argmax token at layer 24 for off-
415 to-think and layer 28 for think-to-off. We then delete every copied state, insert the selected token
416 into the untouched target context, and sample with temperature 0.6, top-p 0.95, and top-k 20. Each
417 prompt and direction uses three seeds with common random numbers across conditions. The design
418 reports 216 direction-labeled analysis rows. Because a canonical reference completion serves as the
419 donor in one direction and the target in the other, these rows correspond to 144 prompt/context/seed
420 generation conditions.

Table 13: Qwen3-4B response format after one residual-selected branch token on 12 disjoint, template-matched prompts under three sampling seeds. The target control inserts the unmodified target mode’s argmax token into the same target context.

Direction	Condition	Observed format	Completed	Reopened think	Mean tokens
Off to think	Donor-token reference	36/36 thinking	36/36	0/36	1976.9
Off to think	One-token mediation	36/36 thinking	36/36	0/36	1809.1
Off to think	Target-token control	36/36 direct	36/36	0/36	224.2
Think to off	Donor-token reference	36/36 direct	36/36	0/36	224.2
Think to off	One-token mediation	36/36 direct	36/36	0/36	218.1
Think to off	Target-token control	36/36 thinking	36/36	0/36	1976.9

421 The copied state selects the donor token on 12/12 prompts in each direction. The 72 mediated
422 continuations then match the donor response format, while all 72 paired target-control rows retain
423 the target format. Every row has exactly one closing think tag and a nonempty answer, and none
424 reaches the 4,096-token limit. The intervention also transfers the large length difference: off-to-
425 think averages 1809.1 tokens versus 224.2 for its direct control, while think-to-off averages 218.1
426 versus 1976.9 for its thinking control. This establishes that the causally selected branch decision is
427 sufficient for the later visible format on this template-matched prompt-value replication. It does not
428 show broad-distribution transfer or that the discarded residual state controls later tokens.

429 F.2 Earlier Qwen generated-prefix diagnostics

430 Generated-prefix tests ask what happens after sampled tokens begin changing the next model state.
431 In a Qwen diagnostic, a vector is learned on 24 public prompts and evaluated on 40 disjoint
432 GSM8K/MATH-500 prompts. It is added for the first generated decision and then for up to 64
433 later decisions before the model continues freely. The selected no-think-to-think vector changes
434 response style and performs favorably on this one held-out set; the reverse row shows that style
435 movement need not improve answers.

Condition	Acc.	Direct	Self-corr.	Tokens	Trunc.
canonical no-think	0.725	0.925	0.075	530.5	0.150
canonical think	0.200	0.125	0.850	1012.8	0.950
no-think → think generated-prefix vector	0.800	0.000	0.000	516.1	0.100
think → no-think generated-prefix vector	0.075	1.000	0.225	1013.0	0.950

The serialized-suffix diagnostic asks whether downstream behavior moves after the prompt-boundary state enters the transient no-think branch. On a separate 24-prompt public subset, canonical Qwen think is much longer and more self-correcting but truncates on 75.0% of prompts; no-think transient states are shorter, less self-correcting, and match no-think-style direct-answer behavior. Accuracy is included to confirm complete responses.

Variant	Think	Direct	Self-corr.	Acc.	Tokens	Trunc.
think suffix	0.750	0.167	0.917	0.417	3585.9	0.750
shared prefix	0.208	0.708	0.500	0.500	2022.2	0.375
no-think preclose	0.000	0.875	0.167	0.708	984.4	0.125
closed, no gap	0.000	0.875	0.167	0.708	1028.0	0.125
full no-think suffix	0.000	0.875	0.250	0.708	1071.7	0.125

F.3 Nemotron generation details

Ten-seed entry-format audit. We first test the opening under ten paired generation/random-vector seed settings, using the same 30 frozen non-math prompts and the same frozen width-2 controller. Each run changes exactly the first generated decision and is capped at 64 tokens. We pre-specify an exact <think> opening as the forward generated-format endpoint. The fixed vector reaches that endpoint on 293/300 generations, versus 0/300 for the norm-matched random vectors. The paired difference is .977. A two-way bootstrap that resamples both prompts and paired run settings gives a 95% interval of [.930, 1.000] from 10,000 resamples. Every individual run scores between 27/30 and 30/30 for the fixed vector, while every random-vector run scores 0/30.

Table 14: Nemotron ten-seed branch-entry audit. “First-step donor win” compares the intervened next-token distribution with the canonical donor and target distributions; the exact-opening row instead evaluates sampled text.

Endpoint	Fixed vector	Random vector	Per-run counts
Exact <think> opening, off to think	293/300	0/300	Fixed 27–30/30; random 0/30
First-step donor win, off to think	300/300	5/300	Fixed 30/30; random 0–2/30
First-step donor win, think to off	300/300	0/300	Fixed 30/30; random 0/30

All 300 fixed off-to-think interventions select <th as the first token. The seven strict opening failures occur during ordinary sampling after that intervened token and produce malformed continuations such as <think offhand or <think off-topic>; they remain counted as failures. They are concentrated on three prompt values: multiturn_02 twice, nonenglish_02 once, and nonenglish_04 four times. We omit a reverse generated-format rate because merely failing to open with <think> does not distinguish a clean direct answer from an unrelated or malformed random-vector output. We instead report the saved first-step donor-win endpoint in both directions. For each prompt, the fixed first-step distribution is deterministic and therefore repeats across the ten run settings; the 300/300 fixed rows in each direction are descriptive, not 300 independent distributional replications. The random-vector controls vary across runs. Reverse fixed generation is also greedy, so its output repeats across seeds. The 64-token cap makes this an entry-format audit; the separate experiment below supplies the 1,024-token completion, length, and repetition evidence.

One fixed-vector step on 30 held-out non-math prompts. We use the frozen width-2 vector learned from the 60 GSM8K/MATH-500 training prompts and apply it for exactly the first generated decision. We then let the model continue freely for up to 1,024 tokens. The evaluation set has 30 previously frozen prompts: six each from five non-mathematical categories. The experiment contains exactly 180 generations: two canonical modes, two fixed-vector directions, and two norm-matched random-vector directions for every prompt.

Table 15: Nemotron generation after one fixed-vector step. “Donor opening” means starting with `<think>` in the forward direction and starting without it in reverse. The random vector is norm matched and uses the same scale and decoding policy.

Condition	Donor opening	Completed	Repetitive	Mean tokens
Canonical off	–	24/30	0/30	643.7
Canonical thinking	–	16/30	0/30	889.1
Off plus thinking vector	29/30	15/30	0/30	863.1
Thinking plus off vector	30/30	12/30	0/30	857.0
Random vector, forward	0/30	24/30	0/30	697.5
Random vector, reverse	0/30	12/30	0/30	915.7

The fixed and random vectors differ on every prompt at the first generated decision in both directions. The fixed vector produces the donor opening on 29/30 forward prompts versus 0/30 for random (paired exact $p = 3.73 \times 10^{-9}$) and 30/30 reverse prompts versus 0/30 for random ($p = 1.86 \times 10^{-9}$). None of the 60 fixed-vector outputs triggers the pre-specified repetition test, and none has low token diversity over its first 64 tokens. The one forward miss begins with the malformed fragment `<think offhand`; two reverse outputs enter a think block later.

The forward effect continues beyond the opening: its completion count is 15/30 versus 16/30 for canonical thinking, and its mean length differs by -26.0 tokens (95% prompt-bootstrap interval $[-78.8, 25.1]$). The reverse intervention selects the direct opening but does not restore the full direct-answer trajectory: it completes 12/30 responses versus 24/30 for canonical off ($p = .00183$) and is 213.3 tokens longer on average (95% interval $[124.5, 308.5]$). This is reliable entry control, not symmetric control of the complete response.

As a stress test, we reapplied the same fixed vector for 64 generated decisions on the same 30 prompts. The first-step records and canonical baselines are identical to the one-step run, but all 30 forward outputs begin with a repeated `<th` fragment and all 30 fixed-vector outputs in each direction fail the carried-forward first-64 diversity test. Nine of 30 reverse outputs also meet the full-output repetition rule. The corruption is caused by repeatedly forcing a prompt-end vector after generation has moved into a different state; it is evidence against using this fixed intervention as a long-horizon controller.

Repeated exact activation copies on six math prompts. The second test uses six public math prompts. At each of the first 64 generated decisions, it recomputes the donor and target trajectories using their respective mode prompts plus the same generated prefix, then replaces the target’s final two residual positions with the matching donor positions. After 64 such decisions it continues freely. Thinking mode used three seeds per prompt, giving 18 thinking runs and 18 off-runs modified with the matching thinking activation. The reverse direction used one run per prompt. Table 16 includes every run, including failures.

Table 16: Nemotron generation after recomputing and reapplying the matching donor tail activation at each of the first 64 decisions, then continuing freely. “Correct (all)” retains the last reported answer even for a repetitive run; “correct (finished)” excludes runs still repeating at 8,192 tokens.

Condition	Finished	Degenerate	Correct (all)	Correct (finished)
Unmodified off mode	6/6	0	4/6	4/6
Unmodified thinking mode	18/18	0	17/18	17/18
Off run given thinking activation	17/18	1	17/18	17/17
Thinking run given off activation	4/6	2	6/6	4/4

The intervention is strongest at response entry and then fades even while the matching copy is reapplied. For off runs given the thinking activation, cumulative donor-win rates across the first 1, 4, 8, 16, 32, and 64 patched decisions are 1.000, .958, .694, .535, .450, and .382. In the reverse direction they are 1.000, .708, .542, .417, .349, and .284. Manual inspection found two reverse runs repeating 477 and 350 boxed answers and one forward run cycling among incompatible interpretations. All five initially capped runs were repeated with an 8,192-token limit and exactly reproduced their original 4,096-token prefixes. Complete modified answers often match the donor mode, but these failures show that repeated exact copying is not reliable long-horizon control.

504 G Plain-language glossary

Term	Intuitive meaning	Exact use in this paper
Thinking switch	The public setting that asks a model to produce a thinking-style response	A chat-template option for Qwen and Gemma; a system instruction for Nemotron
Activation	The model’s internal state at one token and layer	A vector read from the residual stream before the next transformer layer
Prompt end or tail	The point immediately before the model writes its answer	The final one, two, four, or eight input-token positions; width 2 is the main setting
Donor and target	The mode copied from, and the run being changed	The donor activation replaces or shifts the target activation for the same user prompt
Exact patch	Copy the actual internal state from the matching donor run	Used to locate where and through which module the mode signal is carried
505 Fixed mode vector	One reusable average shift learned on other prompts	Mean paired donor-minus-target activation, scaled on training prompts only; 60 fit prompts in the original suites and 12 in the Qwen3 branch-point audit
Donor win	The changed run looks more like the desired mode than its original mode	Its next-token distribution has lower JS distance to the donor than to the target
Effective	A donor win large enough to matter under a fixed threshold	Donor win with JS margin > 0.1 and JS-to-donor < 0.6
Post-attention state / MLP update	The full residual state after attention, versus the MLP’s added update	The former is copied in full; the latter is copied alone, so the asymmetric result does not isolate attention
Low-rank subspace	A small set of shared activation directions	An SVD basis fitted on training differences; unlike the fixed vector, evaluation uses each held-out donor difference’s coordinates

506 NeurIPS Paper Checklist

507 1. Claims

508 Question: Do the main claims made in the abstract and introduction accurately reflect the
509 paper’s contributions and scope?

510 Answer: [Yes].

511 Justification: The abstract and introduction state the positive claim as a prompt-boundary
512 branch-state audit for thinking-mode entry with model-specific residual-tail controllers.
513 The limitations section scopes this claim by separating prompt-boundary branch control
514 from rollout correctness, bidirectional erasure, and contest-math solving.

515 Guidelines:

- 516 • The answer [N/A] means that the abstract and introduction do not include the claims
517 made in the paper.
- 518 • The abstract and/or introduction should clearly state the claims made, including the
519 contributions made in the paper and important assumptions and limitations. A [No] or
520 [N/A] answer to this question will not be perceived well by the reviewers.
- 521 • The claims made should match theoretical and experimental results, and reflect how
522 much the results can be expected to generalize to other settings.
- 523 • It is fine to include aspirational goals as motivation as long as it is clear that these
524 goals are not attained by the paper.

525 2. Limitations

526 Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: The limitations section discusses model-family scope, paired-checkpoint versus same-checkpoint evidence, direction-specific controller asymmetries, Qwen’s mixed broad fixed-vector transfer, generated-prefix diagnostics, and the distinction between prompt-boundary branch-state control and rollout correctness.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper is empirical and does not present theoretical theorems or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: The setup defines the intervention metrics, patch widths, low-rank and fixed-vector fitting procedures, model families, prompt splits, and generated-prefix diagnostic setting; the artifact map points to scripts, row-level JSON files, aggregate JSON files, and run metadata for the reported results.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: The experiments use open model checkpoints and public datasets where applicable. The anonymous supplement contains reproduction scripts, selected row-level and aggregate artifacts, prompt splits, and a README with fast verification commands and model-level reproduction requirements. Written appendices and the checklist are included in the main PDF rather than in the supplement.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).

- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The paper states the 3-fold 40/20 prompt cross-validation, fixed-vector train/eval splits, generated-prefix diagnostic setting, tail width 2, key layer boundaries, prompt counts, and generation caps. The supplement includes the corresponding experiment scripts, prompt splits, aggregate and row-level artifacts, and sanitized run metadata.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes].

Justification: The main low-rank and fixed-vector tables report prompt-bootstrap confidence intervals where space permits, the setup defines the bootstrap procedure, and the text reports finite-suite caveats for exact 60/60 or 0/60 rates. Null opposite-sign, random, wrong-position, and non-tail controls are reported alongside the learned interventions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).

- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes].

Justification: The appendix states that the reported runs used MLX/MLX-LM on an Apple M4 Mac Mini class machine with 16GB unified memory. The included run metadata records start timestamps, platform, model identifier, prompt counts, patch grids or generation caps where applicable, and the paper notes the memory-driven BF16 and generation-cap constraints.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The work studies open model internals and public benchmarks, does not involve human subjects, and does not train or release a new high-risk model.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes].

Justification: The Broader Impacts section discusses the positive auditability benefit and the dual-use risk that better understanding of public thinking switches could improve both oversight and manipulation of thinking-style behavior.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.

- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not release a new pretrained model, scraped dataset, or generative system. It releases analysis scripts and small artifacts derived from open checkpoints.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes].

Justification: The appendix lists the model checkpoints and public branch-audit datasets used, with license notes for Qwen, OLMo, Gemma, LiquidAI, Phi, Nemotron, and math-ai AIME 2025/2026 where available from the source cards. Public generation diagnostics are documented as behavior artifacts for the same intervention family.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.

791 • For existing datasets that are re-packaged, both the original license and the license of
792 the derived asset (if it has changed) should be provided.
793 • If this information is not available online, the authors are encouraged to reach out to
794 the asset’s creators.

795 **13. New assets**

796 Question: Are new assets introduced in the paper well documented and is the documenta-
797 tion provided alongside the assets?

798 Answer: [Yes].

799 Justification: The new assets are experiment scripts and derived activation-analysis artifacts.
800 They are organized by run directory with summaries, sanitized metadata, aggregate files,
801 row-level files, prompt splits, and selected reasoning-mode adapter metadata.

802 Guidelines:

803 • The answer [N/A] means that the paper does not release new assets.
804 • Researchers should communicate the details of the dataset/code/model as part of their
805 submissions via structured templates. This includes details about training, license,
806 limitations, etc.
807 • The paper should discuss whether and how consent was obtained from people whose
808 asset is used.
809 • At submission time, remember to anonymize your assets (if applicable). You can
810 either create an anonymized URL or include an anonymized zip file.

811 **14. Crowdsourcing and research with human subjects**

812 Question: For crowdsourcing experiments and research with human subjects, does the pa-
813 per include the full text of instructions given to participants and screenshots, if applicable,
814 as well as details about compensation (if any)?

815 Answer: [N/A].

816 Justification: The paper does not involve crowdsourcing or human-subject experiments.

817 Guidelines:

818 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
819 with human subjects.
820 • Including this information in the supplemental material is fine, but if the main contri-
821 bution of the paper involves human subjects, then as much detail as possible should
822 be included in the main paper.
823 • According to the NeurIPS Code of Ethics, workers involved in data collection, cura-
824 tion, or other labor should be paid at least the minimum wage in the country of the
825 data collector.

826 **15. Institutional review board (IRB) approvals or equivalent for research with human**
827 **subjects**

828 Question: Does the paper describe potential risks incurred by study participants, whether
829 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
830 approvals (or an equivalent approval/review based on the requirements of your country or
831 institution) were obtained?

832 Answer: [N/A].

833 Justification: The paper does not involve human-subject research and therefore does not
834 require IRB approval.

835 Guidelines:

836 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
837 with human subjects.
838 • Depending on the country in which research is conducted, IRB approval (or equiva-
839 lent) may be required for any human subjects research. If you obtained IRB approval,
840 you should clearly state this in the paper.

- 841 • We recognize that the procedures for this may vary significantly between institutions
842 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
843 guidelines for their institution.
844 • For initial submissions, do not include any information that would break anonymity
845 (if applicable), such as the institution conducting the review.

846 **16. Declaration of LLM usage**

847 Question: Does the paper describe the usage of LLMs if it is an important, original, or
848 non-standard component of the core methods in this research? Note that if the LLM is used
849 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
850 scientific rigor, or originality of the research, declaration is not required.

851 Answer: [N/A].

852 Justification: LLMs were not important, original, or non-standard components of the core
853 method development in this research.

854 Guidelines:

- 855 • The answer [N/A] means that the core method development in this research does not
856 involve LLMs as any important, original, or non-standard components.
857 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
858 not be described.